

Industrial Control
25791

Behzad Ahi **Assignment 2**

Due Date: 1405/02/30

Note: Bonus problems are indicated in ***purple***.
Note: You are expected to upload a PDF file that includes a *detailed derivation* procedure for each question. For the simulation part, along with the essential plots, it is necessary to *append* your *code* at the end of the PDF file. Additionally, provide the *source codes* in a separate folder, where the name of each source code corresponds to the number of the related question.

1 PID Tuning using CHR Method

Tune PID controllers for the given transfer functions using CHR method. The main designing goal is tracking. Design and compare results for no overshoot and 20% overshoot responses.

(a) $G_1(s) = \dfrac{80}{(s+8)^2}\left(\dfrac{1}{s+2} + \dfrac{0.4}{s+0.04}\right)$

(b) $G_2(s) = \dfrac{(s+5)^2}{s(s+2)^2(s+25)}$

2 Simulating System Response

Using the step command in MATLAB, plot the system's response as a function of time.

$$G(s) = \frac{1}{(s+1)^8}$$

Then, obtain the parameters of following models using an arbitrary but different approach for each case.

(a) For a first-order transfer function:

$$G_m(s) = \frac{k}{1+sT}$$

(b) For a second-order model:

$$G_m(s) = \frac{k}{1+sT}e^{-sL}$$

(c) For a fourth-order model:

$$G_m(s) = \frac{k}{(1+sT_1)(1+sT_2)}e^{-sL}$$

The results for each case can be plotted using MATLAB's 'step' function to analyze the system's time-domain response.

3 Method of Moments for System Identification

Here we would like to use MATLAB and find 1st and 2nd order three-parameter model by logging response of system to a known input. Note that in real word condition, we actually do not know anything about the system. The brief explanation of procedure is provided as follows:

1. Generate a swept-frequency cosine signal as the input with following properties:
   - $T_s = 0.01$
   - $T = 25$ (the time period we want to generate the signal)
   - It's frequency increases linearly and it has $f = 0.1$HZ at $t = 0$s and $f = 50$HZ at $t = 25$s

   Suggestion: use chirp command.
2. Now that we have a candidate input, we aim to obtain output. First define the high order system. Then using `lsim` command find the output.
3. Find the moments of input and output. In continuous form we had to calculate integral but here we only need to add elements of function of input and output and multiply it by sample time. You may use sum command like below:
   `sum(f(U)) * Ts % Ts = Sampling time`
4. After calculating the required number of input and output moments, we may find transfer function moments according to formulas provided in slides.
5. Now we can find parameters of model using formulas in the slides. You might want to use `solve` command.
6. At the end plot the step response of the real high-order system and model for the comparison purposes.

Using method of moments, plot the step response of **first and second order three-parameter models** alongside with the step response of the true system for each of the given systems

1. $G_1(s) = 20\dfrac{(s-3)^2}{(s+1)^2(s+0.8)^4}$
2. $G_2(s) = \dfrac{120}{(s+1.6)^6}e^{-s}$

Now generate an arbitrary rich input signal and follow the steps again. Can you find better models than those obtained in previous part?

4 Reaction Curve Based Methods PID Tuning

The Reaction Curve Based Methods are empirical approaches for tuning PID controllers based on the open-loop step response of the process. These methods are particularly useful for plants in process industries that can be satisfactorily described by first-order plus dead-time models of the form:

$$G_o(s) = \frac{K_o e^{-s\tau_o}}{\nu_o s + 1}$$

where K_o is the process gain, τ_o is the delay time, and ν_o is the time constant.

4.1 Obtaining the Process Model

To obtain the parameters of this model, an open-loop experiment is conducted using the following procedure:

1. With the plant in open loop, bring the plant manually to a normal operating point, where the plant output settles at $y(t) = y_o$ for a constant plant input $u(t) = u_o$.
2. At an initial time t_o, apply a step change to the plant input, from u_o to u_f (typically $10 - 20\%$ of full scale).
3. Record the plant output until it settles to the new operating point, obtaining what is known as the process reaction curve.
4. Compute the parameters as follows:

$$\begin{cases} K_o &= \dfrac{y_f - y_o}{u_f - u_o} \\ \tau_o &= t_1 - t_o \\ \nu_o &= t_2 - t_1 \end{cases}$$

   where t_1 is the point where the maximum slope tangent intersects the initial output level, and t_2 is the point where this tangent intersects the final output level.

4.2 Ziegler-Nichols Reaction Curve Method

Ziegler and Nichols proposed controller parameters based on the reaction curve with the objective of achieving a damping ratio that produces a 4:1 decay ratio (the ratio of the first to second peak in the step response). Table 1 shows the recommended settings.

Table 1: Ziegler-Nichols Tuning Using the Reaction Curve

Controller	K_p	T_r	T_d
P	$\dfrac{\nu_o}{K_o \tau_o}$	–	–
PI	$\dfrac{0.9\nu_o}{K_o \tau_o}$	$3\tau_o$	–
PID	$\dfrac{1.2\nu_o}{K_o \tau_o}$	$2\tau_o$	$0.5\tau_o$

A limitation of the Ziegler-Nichols reaction curve method is its sensitivity to the ratio $x = \dfrac{\tau_o}{\nu_o}$ (delay to time constant ratio). The performance varies significantly for different values of this ratio.

4.3 Cohen-Coon Method

To address the limitations of the Ziegler-Nichols method, Cohen and Coon developed controller settings that provide more consistent performance across different values of the ratio $x = \dfrac{\tau_o}{\nu_o}$. Table 2 presents these settings.

Table 2: Cohen-Coon Tuning Using the Reaction Curve

Controller	K_p	T_r	T_d
P	$\frac{\nu_o}{K_o\tau_o}\left[1+\frac{\tau_o}{3\nu_o}\right]$	–	–
PI	$\frac{\nu_o}{K_o\tau_o}\left[0.9+\frac{\tau_o}{12\nu_o}\right]$	$\frac{\tau_o[30\nu_o+3\tau_o]}{9\nu_o+20\tau_o}$	–
PID	$\frac{\nu_o}{K_o\tau_o}\left[\frac{4}{3}+\frac{\tau_o}{4\nu_o}\right]$	$\frac{\tau_o[32\nu_o+6\tau_o]}{13\nu_o+8\tau_o}$	$\frac{4\tau_o\nu_o}{11\nu_o+2\tau_o}$

The Cohen-Coon method provides more homogeneous responses across different values of x compared to the Ziegler-Nichols method. For small values of x, both methods yield similar settings, but the Cohen-Coon method performs more consistently as x increases.

It's important to note that both methods provide only initial tuning parameters. Further refinement is often necessary to achieve optimal performance for specific applications. Modern approaches to PID design, such as those based on model-based techniques, generally provide better performance and are preferable when applicable.

Problems:

(a) Consider a plant with nominal model

$$G_o(s) = \frac{-s+1}{(s+1)^2}.$$

Determine the settings that you would obtain on applying the **Ziegler-Nichols** ultimate gain method.

(b) Consider the model

$$G_o(s) = \frac{-\alpha s+1}{(s+1)(s+3)}.$$

Use the Cohen-Coon tuning strategy to synthesize a PID controller for different values of α in the interval $[0.1,\ 20]$.

(c) Consider a plant with nominal model

$$G_o(s) = \frac{Ke^{-s\tau}}{Ts+1}.$$

Assume you apply the **Cohen-Coon** tuning method.

i) Find the gain and phase margins for different ratios τ/T.

ii) Apply the Ziegler–Nichols and the modified tuning rules to design PI controllers for systems with the transfer functions

$$P_1 = e^{-s}/s, \qquad P_2 = e^{-s}/(s+1), \qquad P_3 = e^{-s}.$$

Compute the stability margins and explore any patterns.

iii) Consider a system with transfer function

$$P(s) = e^{-s}/s.$$

Determine the parameters of P, PI, and PID controllers using Ziegler–Nichols step and frequency response methods. Compare the parameter values obtained by the different rules and discuss the results.

(d) To verify the theoretical concepts presented above, we will implement and analyze the behavior of PID controllers tuned using the reaction curve-based methods. The following MATLAB exercises will provide practical insights into the effectiveness of these tuning strategies under various conditions. Implement the following MATLAB function to identify the process parameters from the step response:

```
function [K_o, tau_o, nu_o] = identifyProcessParams(time, output, inputStep)
    % Extracts first-order plus dead time model parameters from step response data
    %
    % Inputs:
    % time - time vector of the recorded data
    % output - process output vector
    % inputStep - magnitude of the input step change
    %
    % Outputs:
    % K_o - process gain
    % tau_o - delay time
    % nu_o - time constant

    % Calculate steady-state gain
    initialVal = output(1);
    finalVal = output(end);
    K_o = (finalVal - initialVal) / inputStep;

    % Find steepest slope point
    outputDiff = diff(output) ./ diff(time);
    [maxSlope, maxSlopeIdx] = max(outputDiff);

    % Calculate tangent line parameters
    b = output(maxSlopeIdx) - maxSlope * time(maxSlopeIdx);

    % Find intersection points
    t1 = (initialVal - b) / maxSlope;
    t2 = (finalVal - b) / maxSlope;

    % Calculate model parameters
    tau_o = t1 - time(1);
    nu_o = t2 - t1;
end
```

(e) Create a simulation script to compare the closed-loop performance of both tuning methods for a representative process mode

```
% Define the first-order plus dead time process
K_o = 2;
tau_o = 0.5;
nu_o = 3;

% Define the process transfer function
s = tf('s');
G_process = K_o * exp(-tau_o * s) / (nu_o * s + 1);

% Calculate Ziegler-Nichols PID parameters
```

```
Kp_ZN = 1.2 * nu_o / (K_o * tau_o);
Ti_ZN = 2 * tau_o;
Td_ZN = 0.5 * tau_o;

% Calculate Cohen-Coon PID parameters
x = tau_o / nu_o;
Kp_CC = (nu_o / (K_o * tau_o)) * (4/3 + tau_o / (4 * nu_o));
Ti_CC = tau_o * (32 * nu_o + 6 * tau_o) / (13 * nu_o + 8 * tau_o);
Td_CC = 4 * tau_o * nu_o / (11 * nu_o + 2 * tau_o);

% Create PID controllers
C_ZN = Kp_ZN * (1 + 1/(Ti_ZN*s) + Td_ZN*s);
C_CC = Kp_CC * (1 + 1/(Ti_CC*s) + Td_CC*s);

% Create closed-loop systems
CL_ZN = feedback(C_ZN * G_process, 1);
CL_CC = feedback(C_CC * G_process, 1);

% Simulate step responses
t = 0:0.1:30;
figure;
step(CL_ZN, CL_CC, t);
legend('Ziegler-Nichols', 'Cohen-Coon');
title('Comparison of Closed-Loop Step Responses');
grid on;
```

(f) Implement a sensitivity analysis to evaluate how robust the Ziegler-Nichols and Cohen-Coon tuned controllers are to process parameter variations:

```
% Define nominal process parameters
K_o_nom = 2;
tau_o_nom = 0.5;
nu_o_nom = 3;

% Tune controllers for nominal plant
% [Code from Exercise 1 to calculate controller parameters]

% Define parametric uncertainties (%) to be analyzed
param_variations = [-20, -10, 0, 10, 20];

% Initialize arrays to store performance metrics
overshoot_ZN = zeros(length(param_variations), 3);
overshoot_CC = zeros(length(param_variations), 3);
settlingTime_ZN = zeros(length(param_variations), 3);
settlingTime_CC = zeros(length(param_variations), 3);

% Analyze sensitivity to K_o variations
for i = 1:length(param_variations)
    % Vary process gain
    K_o_var = K_o_nom * (1 + param_variations(i)/100);

    % Define varied process
    G_var = K_o_var * exp(-tau_o_nom * s) / (nu_o_nom * s + 1);

    % Closed-loop systems with fixed controllers
    CL_ZN_var = feedback(C_ZN * G_var, 1);
    CL_CC_var = feedback(C_CC * G_var, 1);

    % Calculate performance metrics
    step_info_ZN = stepinfo(CL_ZN_var);
    step_info_CC = stepinfo(CL_CC_var);

    % Store results
    overshoot_ZN(i,1) = step_info_ZN.Overshoot;
    overshoot_CC(i,1) = step_info_CC.Overshoot;
    settlingTime_ZN(i,1) = step_info_ZN.SettlingTime;
```

```
    settlingTime_CC(i,1) = step_info_CC.SettlingTime;
end

% Repeat similar analyses for tau_o and nu_o variations
% [Code to analyze sensitivity to delay and time constant variations]

% Plot results
figure;
subplot(2,3,1);
plot(param_variations, overshoot_ZN(:,1), 'b-o', param_variations, overshoot_CC(:,1), 'r-s');
title('Overshoot vs. Gain Variation');
legend('Ziegler-Nichols', 'Cohen-Coon');
grid on;

% [Additional plots for other metrics and parameters]
```

(g) Analyze the stability margins (gain margin and phase margin) for both tuning methods across a range of $x = \tau_o/\nu_o$ values to determine which method provides better robustness:

```
% Define range of x = tau_o/nu_o values to analyze
x_values = linspace(0.1, 1, 10);

% Initialize arrays for storing stability margins
GM_ZN = zeros(size(x_values));
PM_ZN = zeros(size(x_values));
GM_CC = zeros(size(x_values));
PM_CC = zeros(size(x_values));

% Fixed parameters
K_o = 1;
nu_o = 1;

for i = 1:length(x_values)
    % Calculate tau_o for given x value
    tau_o = x_values(i) * nu_o;

    % Define process transfer function
    G_process = K_o * exp(-tau_o * s) / (nu_o * s + 1);

    % Calculate controller parameters
    % [Code to calculate ZN and CC parameters]

    % Create controllers
    C_ZN = Kp_ZN * (1 + 1/(Ti_ZN*s) + Td_ZN*s);
    C_CC = Kp_CC * (1 + 1/(Ti_CC*s) + Td_CC*s);

    % Calculate stability margins
    [GM_ZN(i), PM_ZN(i)] = margin(C_ZN * G_process);
    [GM_CC(i), PM_CC(i)] = margin(C_CC * G_process);

    % Convert gain margins from ratio to dB
    GM_ZN(i) = 20*log10(GM_ZN(i));
    GM_CC(i) = 20*log10(GM_CC(i));
end

% Plot results
figure;
subplot(2,1,1);
plot(x_values, GM_ZN, 'b-o', x_values, GM_CC, 'r-s');
title('Gain Margin vs. \tau_o/\nu_o Ratio');
xlabel('\tau_o/\nu_o');
ylabel('Gain Margin (dB)');
legend('Ziegler-Nichols', 'Cohen-Coon');
grid on;

subplot(2,1,2);
```

```
plot(x_values, PM_ZN, 'b-o', x_values, PM_CC, 'r-s');
title('Phase Margin vs. \tau_o/\nu_o Ratio');
xlabel('\tau_o/\nu_o');
ylabel('Phase Margin (degrees)');
legend('Ziegler-Nichols', 'Cohen-Coon');
grid on;
```

(h) Implement a simulation to evaluate the disturbance rejection capabilities of controllers tuned with both methods:

```
% Define system and controllers as in previous exercises

% Create Simulink model to test disturbance rejection
% (Alternatively, implement in MATLAB code using lsim)

% Define input signal with step change and disturbance
t = 0:0.1:40;
r = ones(size(t)); % Reference input (step)
d = zeros(size(t));
d(200:end) = 0.3; % Disturbance entering at t=20

% Simulate closed-loop responses with disturbance
[y_ZN, t_ZN] = lsim(CL_ZN, r, t) + lsim(feedback(G_process, C_ZN), d, t);
[y_CC, t_CC] = lsim(CL_CC, r, t) + lsim(feedback(G_process, C_CC), d, t);

% Plot responses
figure;
plot(t, r, 'k--', t, y_ZN, 'b-', t, y_CC, 'r-');
legend('Reference', 'Ziegler-Nichols', 'Cohen-Coon');
title('Disturbance Rejection Comparison');
grid on;
```

(i) Analyze situations where the reaction curve-based tuning methods might fail or provide poor performance, such as for highly oscillatory, integrating, or nonlinear processes:

```
% Define a set of challenging processes
s = tf('s');

% Oscillatory process
G_oscillatory = 1 / (s^2 + 0.1*s + 1);

% Integrating process
G_integrating = 1 / (s * (s + 1));

% Process with inverse response
G_inverse = (-s + 5) / (s^2 + 3*s + 5);

% Obtain step responses
t = 0:0.1:50;
figure;
subplot(3,1,1);
step(G_oscillatory, t);
title('Oscillatory Process Step Response');
grid on;

subplot(3,1,2);
step(G_integrating, t);
title('Integrating Process Step Response');
grid on;

subplot(3,1,3);
step(G_inverse, t);
```

```
title('Process with Inverse Response');
grid on;

% Attempt to identify FOPDT parameters for each process
% [Code to extract FOPDT parameters from step responses]

% Discuss limitations and potential solutions
disp('Limitations of Reaction Curve-Based Methods:');
disp('1. Oscillatory processes: Parameters are difficult to identify accurately');
disp(' Solution: Consider using frequency response methods instead');
disp('2. Integrating processes: Standard FOPDT model is inadequate');
disp(' Solution: Use integrator plus delay model and specialized tuning rules');
disp('3. Processes with inverse response: Simple FOPDT model cannot capture dynamics');
disp(' Solution: Use higher-order models or model-based design approaches');
```

5 PID Synthesis Through Transfer Function Matching

In control system design, we often encounter different representations of controllers that are mathematically equivalent. Any controller of the form

$$C(s) = \frac{n_2 s^2 + n_1 s + n_0}{d_2 s^2 + d_1 s}$$

is equivalent to a PID controller in the standard form

$$C_{PID}(s) = K_p + \frac{K_I}{s} + \frac{K_D s}{\tau_D s + 1}$$

where the parameters are related by specific formulas.

Problem: Consider a control system with a plant having the transfer function

$$G(s) = \frac{3}{(s+1)(2s+3)}$$

(a) Prove that the general controller form $C(s) = \frac{n_2 s^2 + n_1 s + n_0}{d_2 s^2 + d_1 s}$ is equivalent to the standard PID form $C_{PID}(s) = K_p + \frac{K_I}{s} + \frac{K_D s}{\tau_D s + 1}$ by deriving the relationships between their parameters. Your proof should establish the following relationships:

$$\begin{cases} K_p &= \dfrac{n_1}{d_1} - \dfrac{n_0 d_2}{d_1^2} \\ K_I &= \dfrac{n_0}{d_1} \\ K_D &= \dfrac{n_2}{d_1^2} - \dfrac{n_1 d_2}{d_1^3} + \dfrac{n_0 d_2^2}{d_1^3} \\ \tau_D &= \dfrac{d_2}{d_1} \end{cases}$$

(b) A controller with the transfer function

$$C(s) = \frac{5s^2 + 12s + 8}{2s^2 + 7s} \tag{1}$$

has been designed for the plant. Express this controller in the standard PID form by determining the values of K_p, K_I, K_D, and τ_D using the relationships you proved in part (a).

(c) Verify your answer to part (b) by expanding the standard PID form with your calculated parameters and showing that it is identical to the original controller.

(d) If the controller is modified to $\tilde{C}(s) = \dfrac{5s^2 + 12s + 8}{2s^2 + 7s + \varepsilon}$ where ε is a small positive number, discuss how this affects the PID interpretation. Is this still a standard PID controller? Justify your answer.

(e) Consider a family of controllers given by $C_\alpha(s) = \dfrac{\alpha s^2 + (\alpha + 7)s + 8}{2s^2 + 7s}$ where α is a positive parameter. Find the range of values for α such that the corresponding PID controller has positive values for all its parameters (K_p, K_I, and K_D).

6 Setpoint Tracking via Pole Placement

Consider the following first-order plant:

$$G(s) \;=\; \frac{1}{s+1}$$

It is desired to design a controller $C(s)$ such that:

- The closed-loop poles are placed at $s = -2 \pm 2j$
- The steady-state error to a **ramp input** $D(s) = \dfrac{1}{s^2}$ is **zero**.

hint: use the augmented transfer function.

Problem:

(a) State the minimum order m of the controller $C'(s) = P(s)/L(s)$ required so that the degree condition $m \geq \deg A' - 1$ is satisfied.

(b) Write the desired closed-loop characteristic polynomial. The closed-loop poles of the augmented system must include the desired poles $s = -2 \pm 2j$. Place the extra poles at s = -5 (why?).

(c) Verify that Sylvester matrix will be non-singular. Write the polynomial identity:

$$A'(s)\,L(s) \;+\; B'(s)\,P(s) \;=\; \Delta_d(s)$$

Expand both sides, collect terms by powers of s, and set up the linear system $M\boldsymbol{\theta} = \mathbf{x}$. Write out the matrix M and the vector $\mathbf{x}$ explicitly.

Solve the system $M\boldsymbol{\theta} = \mathbf{x}$ and obtain the coefficients of $L(s)$ and $P(s)$. Write the final controller.

(d) Using the Final Value Theorem (applied to the appropriate disturbance-to-output transfer function), argue why the steady-state error due to the ramp input is zero.

(e) Would a simple proportional controller $C(s) = K$ be sufficient to satisfy the stated task? Explain briefly.

(f) Build the closed-loop system in MATLAB or simulink and extract its poles. Confirm they match $\Delta_d(s)$.

(g) Apply the ramp input and observe the output. Show that the steady-state error converges to zero.

(h) Repeat the simulation replacing $C(s)$ with the proportional controller. Show that the error does *not* converge to zero.

(i) Apply ramp disturbance(starting from $t = 3s$) at the **plant** input and observe the output. Has the disturbance been rejected or not? explain why.

7 Windup and Anti-Windup

Integral windup is a common phenomenon in control systems. Generally, it happens when the integrator continues to integrate error that the actuator cannot correct (e.g., due to presence of saturation). In such cases, we might see large overshoot and slow recovery. In this problem, you will investigate the windup phenomenon and implement several anti-windup strategies.

Consider a plant with transfer function:

$$P(s) = \frac{1}{(s+1)}$$

The actuator is subject to saturation:

$$u = \text{sat}(u_c), \quad |u| \leq 2$$

A PI controller of the form

$$C(s) = K_p + \frac{K_i}{s}$$

is used.

Problems:

(a) set controller gains $K_p = 5$ and $K_i = 15$ so that the closed-loop system bandwidth is relatively high (aggressive tuning).

- Apply a step input of amplitude 1 starting from $t = 1s$.
- Simulate the closed-loop system with actuator saturation.
- Plot:
  - Output $y(t)$
  - Control signal before saturation $u_c(t)$
  - Saturated control signal $u(t)$

Describe the observed behavior and explain why integral windup occurs.

(b) Back-Calculation Anti-Windup: Implement the back-calculation method using:

$$\dot{x}_I = e + k_{aw}(u - u_c)$$

where x_I is the integrator state and $k_{aw} > 0$ is the anti-windup gain.

- Simulate the system for at least two different values of k_{aw}.

Explain the effect of increasing and decreasing k_{aw} on system performance.

(c) Conditional Integration: In this approach, the saturated controller acts as an anti-windup mechanism by disabling the integration term whenever the controller is saturated in a direction that would further increase the control signal.

In other words, if the control signal u is saturated, the integration is halted if the product of the control signal and the error (ue) is positive. For instance, if the control signal is at the upper saturation limit, integration must stop if the error is positive ($e > 0$). Conversely, integration stops if the control signal is at the lower saturation limit and the error is negative ($e < 0$). If the error is positive while the control signal is at the lower saturation limit, integration continues.

- Simulate the system and compare the results with the previous method.

Discuss advantages and possible drawbacks of this approach.

(d) The Simulink **PID Controller** block provides built-in anti-windup mechanisms. Below is the step-by-step procedure to implement and configure this feature:

Step 1: Add the PID Controller Block and Set Up the Control Loop.

Step 2: Since anti-windup is only meaningful when the actuator has limits, you must define these limits within the block:

- Double-click the block to open the **Block Parameters** window.
- Navigate to the **Output Saturation** tab.
- Check the box **Limit output**.
- Enter the **Upper limit** and **Lower limit**.

Step 3: In the same parameters window, locate the **Anti-windup method** drop-down menu. You can choose between two primary methods, **Back-calculation** and **Clamping**.

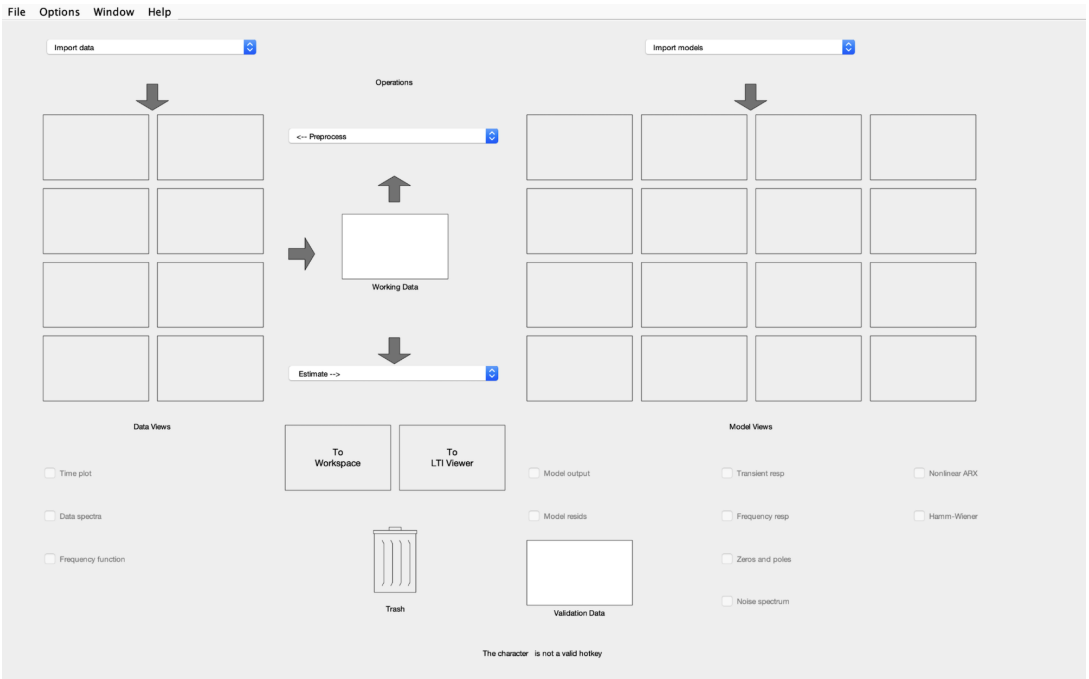
Simulate the system and compare the results with `part b` and `part c`.

(e) Conceptual Analysis: Answer the following questions:

- Is the windup problem unique to PID controllers, or can it occur in other control strategies?
- Why does increasing the closed-loop bandwidth make windup more severe?
- Why does the integrator state become problematic during saturation?
- How does each anti-windup method modify the integrator dynamics?

8 MATLAB Identification Toolbox

MATLAB has different toolboxes. Among them Ident toolbox is commonly used to find, estimate and fit the best pattern to a set of data consisting of input and corresponding output. It has the following shape:



On the up-left-hand side you will see import data. You can choose between different type of data. We usually use time-domain data. After entering data remember to set the start time and sample time. Here set that to $T_{start} = 1$ and $T_{sampling} = 0.1$. In the preprocess part, we can choose to apply different operations that we find necessary on data. Operations like Remove mean, Remove trends, Filter, etc. here we use Remove mean option. Now data is ready. We move to the next step. There is an Estimate option in the screen. Opening it leads to a list of different options containing transfer function, state space representation, non-linear models, polynomial models, etc. we would like to find the best transfer function that estimates the data. Choosing transfer function option, we find ourselves in a new page asking for number of poles and zeros of the desired model. After setting the numbers by clicking the estimate button in the same page, the process begins. After a few seconds you can find your model on the up-right-hand side of the screen.
For the given data (`Ident.mat`) use Ident toolbox and estimate the transfer function of corresponding system. Report achieved indices that specify the goodness of fitness.

9 Model Order Reduction

MATLAB provides powerful tools for model order reduction, allowing you to simplify complex systems while preserving essential dynamic characteristics. In this exercise, you are required to use the **Model Reducer app** in MATLAB to reduce the order of a high-order system.

The system is defined by the following transfer function:

$$G(s) = \frac{(0.2s+1)(s+0.7)(s+1)(s+2)}{(25s+1)(12s+1)(8s+1)(3s+1)(s+0.4)(s+0.2)}$$

1. Use the **Model Reducer** app to simplify the transfer function $G(s)$.
2. Apply mode selection by choosing appropriate lower and upper cutoff frequencies to obtain an optimal lower-order model.
3. Compare the frequency response of the reduced model with the original system over the specified frequency range:
$$0.05 \leq \omega \leq 15$$
4. Analyze the performance of the reduced model and discuss any loss of accuracy.

10 Curve Fitting Toolbox Task

The MATLAB Curve Fitting Toolbox is a powerful tool designed to facilitate the process of fitting curves and surfaces to data. It provides a comprehensive set of functions and tools for analyzing and modeling data, including various algorithms for curve fitting, interpolation, smoothing, and surface fitting. In this task, you are required to work with multiple models and datasets to evaluate the performance of curve fitting techniques.

Suppose we have several sets of experimental data representing different types of biological growth processes over time. You are required to fit these datasets to various growth models and compare their effectiveness. The growth models to be considered include:

1. **Logistic Growth Model:**
$$P(t) = \frac{K}{1 + \frac{K-P_0}{P_0}e^{-rt}}$$
2. **Gompertz Growth Model:**
$$P(t) = K \exp\left(-e^{-r(t-t_0)}\right)$$
3. **Richards' Growth Model (Generalized Logistic):**
$$P(t) = \frac{K}{\left(1 + \alpha e^{-rt}\right)^{1/\nu}}$$

4. **Bertalanffy Growth Model:**

$$P(t) = K\left(1 - e^{-rt}\right)^3$$

Problem

(a) Load the datasets provided in `GrowthData1.csv`, `GrowthData2.csv`,`GrowthData3.csv` and `GrowthData4.csv`.

(b) For each dataset, fit the data to the four growth models listed above using the **Curve Fitting Toolbox**.

(c) Provide plots of the fitted curves and discuss their accuracy.

(d) Compare the models by calculating the goodness-of-fit metrics (e.g., R-squared, RMSE).

(e) Analyze which model is most appropriate for each dataset and explain why.

11 PID Controller Optimization

PID controller tuning is a critical aspect of control systems engineering. While classical methods such as Ziegler-Nichols offer initial parameter estimates, modern optimization methods (e.g., Genetic Algorithms) provide tailored solutions based on specific performance objectives and constraints. This document considers the design of a PIDF controller for a chemical reactor system using GA-based optimization.

A standard PID controller consists of proportional, integral, and derivative components. However, because the derivative term may amplify high-frequency noise, a first-order low-pass filter is added, yielding the PIDF controller:

$$C(s) = K_p + \frac{K_i}{s} + \frac{K_d s}{T_f s + 1},$$

where K_p is the proportional gain, K_i is the integral gain, K_d is the derivative gain, and T_f is the filter time constant. The filter reduces the sensitivity of the derivative term to noise while preserving its ability to improve transient response.

Problem: Consider a chemical reactor system modeled by the following transfer function:

$$G(s) = \frac{2e^{-0.3s}}{(4s+1)(3s+1)}.$$

The goal is to design a PIDF controller that optimizes performance (transient and steady-state) while ensuring stability and robustness.

(a) Derive the closed-loop transfer function of the system with the PIDF controller structure. Explain the role of the filter term in the derivative component.

(b) Analyze the system's stability characteristics and determine the theoretical constraints on controller parameters to ensure closed-loop stability.

(c) Compare and contrast at least three different cost functions that could be used for optimizing the PIDF controller. Discuss how each cost function influences the resulting closed-loop response characteristics. Some common cost functions include:

- **Integral of Absolute Error (IAE):** Measures overall system error magnitude, emphasizing steady-state performance.

$$IAE = \int_0^\infty |e(t)|dt$$

- **Integral of Squared Error (ISE):** Penalizes larger errors more heavily, promoting smooth error convergence.

$$ISE = \int_0^\infty e^2(t)dt$$

- **Integral of Time-Weighted Absolute Error (ITAE):** Prioritizes error reduction in the transient phase, improving settling time and reducing overshoot.

$$ITAE = \int_0^\infty t|e(t)|dt$$

- **Integral of Time-Weighted Squared Error (ITSE):** Similar to ITAE but penalizes larger errors more, improving transient response smoothness.

$$ITSE = \int_0^\infty te^2(t)dt$$

- **Root Mean Square Error (RMSE):** Measures the square root of the mean of squared errors, balancing steady-state and transient performance.

$$RMSE = \sqrt{\frac{1}{T}\int_0^T e^2(t)dt}$$

- **Maximum Overshoot (MPO):** Measures the highest peak deviation from the steady-state value.

$$MPO = \max\left(\frac{|y_{max} - y_{ss}|}{y_{ss}} \times 100\%\right)$$

Explain how each of these cost functions could be applied in the optimization process and how they might affect transient and steady-state performance.

(d) Explain the advantages and limitations of using genetic algorithms for PID tuning compared to classical methods like Ziegler-Nichols and Cohen-Coon.

(e) Develop a mathematical expression for a cost function that balances the following requirements:

- Minimize rise time
- Minimize overshoot (less than 5%)
- Guarantee steady-state error less than 1%
- Ensure control signal remains within constraints: $-5 \leq u(t) \leq 5$
- Minimize control effort

(f) Create a MATLAB simulation model to evaluate PID controller performance for the given system. The following MATLAB code snippet illustrates one approach:

```
function J = objectiveFunction(K)
    % Extract PID parameters
    Kp = K(1);
    Ki = K(2);
    Kd = K(3);

    % System model
    num = 2;
    den = conv([5 1], [2 1]);
    G = tf(num, den, 'InputDelay', 0.2);

    % Controller design (PIDF structure)
    % Controller coefficients incorporate a derivative filter.
```

```
    numC = [Kd, Kp, (Ki*0.05 + Kp), Ki];
    denC = [0.05, 1, 0];
    C = tf(numC, denC);

    % Closed-loop system
    CL = feedback(C * G, 1);

    % Simulation parameters
    Ts = 0.01;
    Tf = 30;
    t = 0:Ts:Tf;
    r = ones(size(t)); % Step reference

    % Simulate system response
    [y, t] = step(CL, t);

    % Compute error and control signal
    e = r - y;
    u = lsim(C, e, t);

    % Performance metrics
    ISE = sum(e.^2) * Ts;
    ITSE = sum((t' .* e).^2) * Ts;
    control_effort = sum(u.^2) * Ts;

    % Penalize control signal constraint violations
    u_max_violation = sum(max(0, abs(u) - 5).^2) * Ts;

    % Calculate rise time and overshoot
    [~, idx_90] = min(abs(y - 0.9));
    rise_time = t(idx_90);
    overshoot = max(0, max(y) - 1);
    overshoot_penalty = 0;
    if overshoot > 0.05
        overshoot_penalty = 100 * (overshoot - 0.05)^2;
    end

    % Composite cost function weights
    w1 = 1; % Weight for ITSE (tracking error)
    w2 = 0.1; % Weight for control effort
    w3 = 100; % Weight for constraint violations
    w4 = 10; % Weight for rise time
    w5 = 1000; % Weight for overshoot penalty

    J = w1 * ITSE + w2 * control_effort + w3 * u_max_violation + ...
        w4 * rise_time + w5 * overshoot_penalty;
end
```

(g) Use MATLAB's **Genetic Algorithm** toolbox (`ga` function) to find optimal PID parameters. Compare your results with at least two other optimization algorithms (e.g., `particleswarm`, `fmincon`, `simulannealbnd`). The script below demonstrates how to run the `ga`:

```
% Define bounds for PID parameters [Kp, Ki, Kd]
lb = [0, 0, 0]; % Lower bounds
ub = [10, 10, 10]; % Upper bounds

% Configure GA options
options = optimoptions('ga', 'PopulationSize', 50, ...
                    'MaxGenerations', 100, ...
                    'PlotFcn', @gaplotbestf, ...
                    'Display', 'iter');

% Run the Genetic Algorithm
[K_opt, J_opt] = ga(@objectiveFunction, 3, [], [], [], [], lb, ub, [], options);

% Display optimal PID parameters
```

```
fprintf('Optimal PID Parameters:\n');
fprintf('Kp = %.4f\n', K_opt(1));
fprintf('Ki = %.4f\n', K_opt(2));
fprintf('Kd = %.4f\n', K_opt(3));
fprintf('Optimal Cost: %.4f\n', J_opt);

% Define the system model
num = 2;
den = conv([5 1], [2 1]);
G = tf(num, den, 'InputDelay', 0.2);

% Construct the optimized controller
numC = [K_opt(3), K_opt(1), (K_opt(2)*0.05 + K_opt(1)), K_opt(2)];
denC = [0.05, 1, 0];
C = tf(numC, denC);

% Form the closed-loop system
CL = feedback(C * G, 1);

% Simulate and plot the step response
t = 0:0.01:30;
figure;
step(CL, t);
title('Step Response with Optimized PID Controller');
grid on;

% (Additional code for disturbance simulation can be added here)
```

(h) Study the effect of changing the population size and generation number on the optimization results.

(i) Verify the robustness of your optimal controller by analyzing:

- Step response characteristics (rise time, settling time, overshoot)
- Response to disturbances (add a step disturbance of magnitude 0.5 at $t = 10$ [s])
- Sensitivity to parameter variations (vary the time constant and delay by $\pm 20\%$)

(j) Implement the optimized controller in a Simulink model that includes:

- Saturation blocks to enforce control signal constraints
- Measurement noise (band-limited white noise with power 0.001)
- Input disturbances

12 PID Tuning with Derivative on Output and Set-point Weighting

Advanced PID controller structures often include modifications such as derivative action on the output signal rather than the error signal, and set-point weighting to separately optimize set-point tracking and disturbance rejection. These modifications help address practical issues in control system implementation, such as derivative kick, measurement noise sensitivity, and conflicting performance objectives.

Consider a temperature control system for a heating process modeled by:

$$G(s) = \frac{1}{(s+2)(8s+1)}$$

The system is subject to:

- **Measurement noise:** high-frequency random fluctuations
- **Input disturbances:** occasional step changes (± 0.2) at the process input
- **Parameter variations:** time constants may vary by $\pm 15\%$ due to environmental factors

Problem:

(a) Derive the transfer function for a modified PID controller with the following structure:

$$u(t) = K_p(br(t) - y(t)) + K_i \int_0^t (r(\tau) - y(\tau))d\tau - K_d \frac{dy(t)}{dt}$$

where b is the set-point weighting parameter for the proportional term.

(b) Derive the closed-loop transfer functions:

- From reference $r(t)$ to output $y(t)$
- From reference $r(t)$ to control signal $u(t)$
- From disturbance $d(t)$ to output $y(t)$
- From measurement noise $n(t)$ to output $y(t)$
- From measurement noise $n(t)$ to control signal $u(t)$

(c) Analyze how the parameter b affects the reference tracking performance versus disturbance rejection capabilities. What is the theoretical optimal value of b for pure disturbance rejection?

(d) Compare the sensitivity of the control signal to measurement noise between:

- Standard PID with derivative on error
- Modified PID with derivative on output

(e) Explain the concept of derivative filtering in PID controllers. Derive the modified transfer function when a first-order filter with time constant τ is applied to the derivative term:

$$D(s) = \frac{K_d s}{1 + \tau s}$$

How does τ affect the noise sensitivity and control performance?

(f) Implement a MATLAB/Simulink model for the system with:

i) Standard PID controller (derivative on error)

ii) Modified PID controller (derivative on output)

iii) Set-point weighting capability (adjustable b parameter)

iv) Derivative filter (adjustable τ parameter)

v) Measurement noise source

vi) Disturbance input

```
% Implementation guideline for Problem (f)
% Create a new model
mdl = 'PIDControlSystem';
new_system(mdl);
open_system(mdl);

% Add blocks
% Process model, reference input, disturbance, noise
% Controllers (standard PID and modified PID)
% Set-point weighting and derivative filtering implementations

% Connect blocks and configure simulation parameters
```

(g) Develop an optimization framework in MATLAB to tune the PID parameters (K_p, K_i, K_d), set-point weighting parameter (b), and derivative filter constant (τ) simultaneously. Use a multi-objective approach to balance:

- Tracking performance (minimize ISE or ITSE for step reference)
- Disturbance rejection (minimize IAE or ITAE for step disturbance)
- Noise sensitivity (minimize variance of control signal due to measurement noise)
- Control effort (minimize the integral of squared control signal)

```
% Multi-objective function
function [J1, J2] = multiObjectiveFunction(x)
    Kp = x(1);
    Ki = x(2);
    Kd = x(3);
    b = x(4);
    tau = x(5);

    metrics = evaluateController(Kp, Ki, Kd, b, tau);

    % First objective: tracking performance
    J1 = metrics.modified_PID.ISE + 10*metrics.modified_PID.rise_time + 100*metrics.modified_PID.
        overshoot;

    % Second objective: disturbance rejection and noise sensitivity
    J2 = metrics.modified_PID.IAE_dist + 0.1*metrics.modified_PID.var_u;
end

% Run optimization
lb = [0, 0, 0, 0, 0.01]; % Lower bounds
ub = [10, 10, 10, 1, 0.5]; % Upper bounds

options = optimoptions('paretosearch', 'ParetoSetSize', 50, ...
                    'Display', 'iter', ...
                    'PlotFcn', {@psplotparetof, @psplotpareto});

[x_pareto, f_pareto] = paretosearch(@multiObjectiveFunction, 5, [], [], [], [], lb, ub, [], options);

% Evaluate and visualize Pareto front solutions
numSolutions = size(x_pareto, 1);
fprintf('Found %d Pareto optimal solutions\n', numSolutions);

% Select representative solutions
% Get knee point of Pareto front (compromise solution)
f_normalized = f_pareto ./ max(f_pareto, [], 1);
distances = sqrt(sum(f_normalized.^2, 2));
[~, knee_idx] = min(distances);
knee_solution = x_pareto(knee_idx, :);

fprintf('Recommended solution (knee point):\n');
fprintf('Kp = %.4f, Ki = %.4f, Kd = %.4f, b = %.4f, tau = %.4f\n', ...
```

```
    knee_solution(1), knee_solution(2), knee_solution(3), knee_solution(4), knee_solution(5));

% Evaluate and visualize the knee solution
evaluateController(knee_solution(1), knee_solution(2), knee_solution(3), knee_solution(4), ...
    knee_solution(5));
```

(h) Perform a parametric study to analyze:

i) Effect of varying b from 0 to 1 on step response and disturbance rejection

ii) Effect of varying τ from 0.01 to 0.5 on noise sensitivity

iii) Impact of process parameter variations on control performance

```
% Parametric study template for Problem (i)
function performParametricStudy()
    % b parameter sweep
    b_values = 0:0.1:1;
    for i = 1:length(b_values)
        b = b_values(i);
        % Fixed values for other parameters
        Kp = 2; Ki = 0.5; Kd = 1; tau = 0.1;
        metrics(i) = evaluateController(Kp, Ki, Kd, b, tau);

        % Store key performance metrics
        ISE_values(i) = metrics(i).modified_PID.ISE;
        IAE_dist_values(i) = metrics(i).modified_PID.IAE_dist;
    end

    % Plot results
    figure;
    plot(b_values, ISE_values, 'b-o', b_values, IAE_dist_values, 'r-s');
    grid on;
    xlabel('Set-point Weighting Parameter (b)');
    ylabel('Performance Metrics');
    legend('ISE (Tracking)', 'IAE (Disturbance Rejection)');
    title('Effect of Set-point Weighting on Performance');

    % Similar implementations for tau parameter sweep
    % and process parameter variations
end
```